HTML

From Basic to Advanced

A Complete Student's Reference Guide

Chapters 1–41

Foundations

Structure & Semantics

Linking & Navigation

Tables & Lists

Media & Graphics

Forms & User Input

Advanced Features

Specialized Markup

Foundations

Focus: Core HTML structure & text basics

Chapter 1: Getting Started with HTML

Definition

HTML (HyperText Markup Language) is the standard markup language for creating web pages. It uses tags to structure content — not to style it. Styling is handled by CSS.

Key Points

- Element = opening tag + content + closing tag: Hello
- Void elements have no closing tag: <img>,
, <input>
- HTML5 is the current standard — always use <!DOCTYPE html>

Code Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is my first HTML page.</p>
</body>
</html>
```

📖 **Tip:** *Save as .html and open in any browser. The <head> holds metadata; <body> holds visible content.*

Chapter 2: Doctypes

Definition

The DOCTYPE declaration tells the browser which version of HTML the document uses. It must be the very first line of an HTML file (before the <html> tag). Without it, browsers enter "quirks mode" and may render incorrectly.

Code Example

```
<!-- HTML5 (recommended — use this always) -->
<!DOCTYPE html>


<!-- HTML 4.01 Strict -->
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
  "http://www.w3.org/TR/html4/strict.dtd">


<!-- DOCTYPE is case-insensitive -->
<!DOCTYPE html>
<!doctype html>
<!DocType Html>
```

🕮 **Tip:** *Always use <!DOCTYPE html> for new projects. HTML5 DOCTYPE is the simplest and works with all modern browsers.*

Chapter 3: Paragraphs

Definition

<p> defines a paragraph of text. The browser adds spacing before and after.
 inserts a line break without starting a new paragraph. <pre> preserves whitespace and line breaks exactly as written.

Code Example

```
<p>This is the first paragraph. It flows naturally.</p>
<p>This is the second paragraph.</p>

<!-- Line break (no new paragraph) -->
<p>Line one.<br>
Line two on same paragraph.</p>

<!-- Pre-formatted text -->
<pre>
  Name: Alice
  Score: 98
  Grade: A
</pre>
```

🕮 **Tip:** *Never use
 tags just to add vertical space — use CSS margin/padding instead.
 is for intentional line breaks in content.*

Chapter 4: Headings

Definition

HTML provides 6 heading levels: <h1> (most important) through <h6> (least important). Headings define the document outline and are critical for accessibility and SEO. Use them in hierarchical order.

Code Example

```
<h1>Page Title (only one per page)</h1>
<h2>Main Section</h2>
<h3>Subsection</h3>
<h4>Sub-subsection</h4>
<h5>Minor heading</h5>
<h6>Smallest heading</h6>
```

🕮 **Tip:** *Never skip heading levels (e.g., jumping from h1 to h4). Screen readers use headings for navigation. Only one <h1> per page.*

Chapter 5: Text Formatting

Definition

HTML provides semantic text formatting tags: (bold, important), (italic, emphasis), <mark> (highlight), <del> (deleted), <ins> (inserted), <sup>/<sub> (super/subscript), and more.

Code Example

```
<strong>Semantically important text</strong>
<em>Emphasized text (stress)</em>
```

```
<mark>Highlighted text</mark>
<p>Price: <del>$50</del> <ins>$35</ins></p>
<p>E = mc<sup>2</sup></p>
<p>H<sub>2</sub>O is water</p>
<abbr title="HyperText Markup Language">HTML</abbr>
```

Tip: *Use and for semantic meaning. Use <b> and <i> only for stylistic text with no semantic importance.*

Chapter 6: Comments

Definition

HTML comments are notes in your code that the browser ignores entirely. They are visible in the page source but do not render on screen.

Code Example

```
<!-- Single line comment -->
<!--
  Multi-line comment:
  This entire section is under construction.
-->

<h1>Hello <!-- this text is hidden --> World</h1>

<!-- Commenting out code temporarily: -->
<!-- <p>This paragraph is disabled for now</p> -->
```

Tip: *Comments cannot be nested. Do not put -- inside a comment body.*

Chapter 7: Character Entities

Definition

Character entities are special codes used to display reserved HTML characters (like < and >) or symbols not on a standard keyboard. They start with & and end with ;.

Code Example

```
<p>5 &lt; 10 and 10 &gt; 5</p>
<p>Fish &amp; Chips</p>
<p>&quot;Quoted text&quot;</p>
<p>10 kg</p>
<p>&copy; 2024 My Company</p>
<p>&mdash; em dash &ndash; en dash</p>
```

Tip: *Always encode < as < and > as > inside HTML content to avoid browser parsing errors.*

Structure & Semantics

Focus: Organizing content meaningfully

Chapter 8: Div Element

Definition

The element is a generic block-level container with no inherent semantic meaning. Use it to group elements for styling or layout when no semantic element fits.

Code Example

```
<div class="card">
  <h2>Card Title</h2>
  <p>Card content goes here.</p>
</div>

<div class="container">
  <div class="sidebar">Sidebar</div>
  <div class="main-content">Main</div>
</div>
```

Tip: *Prefer semantic elements (<article>, <section>, <nav>, <main>) over when the content has meaning.*

Chapter 9: Sectioning Elements

Definition

HTML5 sectioning elements define the document structure semantically: <header>, <nav>, <main>, <section>, <article>, <aside>, and <footer>. They replace generic wrappers with meaningful markup.

Code Example

```
<body>
  <header>
    <h1>My Blog</h1>
    <nav><a href="/">Home</a></nav>
  </header>
  <main>
    <article>
      <h2>Blog Post Title</h2>
      <section>
        <h3>Chapter 1</h3>
        <p>Content...</p>
      </section>
    </article>
    <aside><h3>Related Posts</h3></aside>
  </main>
  <footer><p>&copy; 2024 My Blog.</p></footer>
</body>
```

Tip: *<main> must appear only once per page. <article> is for independently-distributable content. <section> requires a heading.*

Chapter 10: Classes and IDs

Definition

class selects one or more elements for CSS styling (reusable). id uniquely identifies a single element on the page (must be unique — used for JavaScript and anchor links).

Code Example

```
<div class="card highlight">Styled Card</div>
<div class="card">Another Card</div>
<p id="main-intro">Unique paragraph</p>

<style>
  .card { background: #e3f2fd; padding: 16px; }
  .highlight { border-left: 4px solid blue; }
  #main-intro { font-size: 1.2em; color: darkblue; }
</style>
```

Tip: *An element can have multiple classes (space-separated), but only ONE id. IDs must be unique per page.*

Chapter 11: Global Attributes

Definition

Global attributes can be applied to ANY HTML element. They include id, class, style, title, lang, tabindex, contenteditable, draggable, hidden, and more.

Code Example

```
<p contenteditable="true">Click me to edit this text!</p>
<div hidden>This is not visible on the page.</div>
<img src="logo.png" draggable="true" alt="Drag me">
<abbr title="HyperText Markup Language">HTML</abbr>
<p dir="rtl">This text goes right-to-left.</p>
<span translate="no">Brand Name XYZ</span>
```

Tip: *The style and class attributes are the most commonly used global attributes for applying CSS to individual elements.*

Chapter 12: Data Attributes

Definition

Data attributes (data-*) let you store custom information directly on HTML elements without affecting rendering. JavaScript accesses them via the dataset property.

Code Example

```
<div id="user"
  data-user-id="42"
  data-role="admin"
  data-active="true">
  John Doe
</div>

<script>
  const el = document.getElementById("user");
  console.log(el.dataset.userId); // "42"
  console.log(el.dataset.role);   // "admin"
  el.dataset.active = "false";
```

```
</script>
```

Tip: *data-* names are hyphenated in HTML but camelCase in JavaScript dataset. data-user-id becomes dataset.userId.*

Chapter 13: Tabindex

Definition

The tabindex attribute controls keyboard Tab navigation order. tabindex="0" includes an element in the natural tab flow. tabindex="-1" removes it from tab order (but keeps it focusable via JavaScript).

Code Example

```
<!-- Add non-interactive element to tab order -->
<div tabindex="0" onclick="activate(this)">
  Click or Tab to me
</div>

<!-- Remove from tab order (focusable by JS only) -->
<button tabindex="-1" id="modalClose">Close Modal</button>

<!-- Focusable custom component -->
<div role="button" tabindex="0"
  onclick="doAction()"
  onkeydown="if(event.key==='Enter') doAction()">
  Custom Button
</div>
```

Tip: *Avoid positive tabindex values — they break the natural tab flow and confuse users. Rely on DOM order for focus sequence.*

Linking & Navigation

Focus: Connecting pages & resources

Chapter 14: Anchors and Hyperlinks

Definition

The <a> (anchor) element creates clickable hyperlinks. The href attribute specifies the destination URL, page anchor, or protocol like mailto: or tel:.

Key Points

- href – destination URL
- target – where to open
- rel – relationship
- download – force download

Code Example

```
<a href="https://example.com">Visit Example</a>

<!-- Open in new tab (add rel for security) -->
<a href="https://example.com" target="_blank"
   rel="noopener noreferrer">Open New Tab</a>

<!-- Email link -->
<a href="mailto:hello@example.com?subject=Hi">Send Email</a>

<!-- Download link -->
<a href="file.pdf" download="myfile.pdf">Download PDF</a>
```

📖 **Tip:** *Always add rel="noopener noreferrer" when using target="_blank" to prevent security vulnerabilities.*

Chapter 15: Navigation Bars

Definition

Navigation bars are collections of links that let users move between sections or pages. HTML5 provides the semantic <nav> element.

Code Example

```
<nav role="navigation" aria-label="Main navigation">
  <ul>
    <li><a href="/" aria-current="page">Home</a></li>
    <li><a href="/about">About</a></li>
    <li><a href="/services">Services</a></li>
    <li><a href="/contact">Contact</a></li>
  </ul>
</nav>

<style>
nav ul { list-style:none; display:flex; gap:16px;
```

```
  background:#1565C0; }
nav ul li a { color:white; padding:12px 20px; display:block; }
</style>
```

Tip: *Use aria-current="page" on the active link. Use <nav> only for major navigation blocks.*

Chapter 16: Linking Resources

Definition

The <link> element in the <head> connects external resources (stylesheets, icons, fonts, feeds) to the document. The rel attribute specifies the relationship.

Code Example

```
<head>
  <link rel="stylesheet" href="style.css">
  <link rel="icon" type="image/png" href="/favicon.png">

  <!-- Google Fonts -->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="stylesheet"
    href="https://fonts.googleapis.com/css2?family=Roboto">

  <!-- Performance hints -->
  <link rel="dns-prefetch" href="https://cdn.example.com">
  <link rel="canonical" href="https://example.com/page/">
</head>
```

Tip: *Place all <link> tags inside <head>. dns-prefetch and preconnect speed up external resource loading noticeably.*

Chapter 17: Content Languages

Definition

The lang attribute declares the human language of an element's content using BCP 47 language codes. It helps screen readers pronounce content correctly and improves SEO.

Code Example

```
<html lang="en">

<p lang="fr">Bonjour le monde!</p>

<p lang="en">
  The German word for world is
  <span lang="de">Welt</span>.
</p>

<a href="https://example.fr" hreflang="fr">French version</a>
```

Tip: *Always set lang on the <html> element. This is a WCAG 2.0 accessibility requirement (Success Criterion 3.1.1).*

Tables & Lists

Focus: Structured data presentation

Chapter 18: Tables

Definition

HTML tables display two-dimensional tabular data in rows and columns. Key elements: <table>, <thead>, <tbody>, <tfoot>, <tr>, <th>, <td>, and <caption>. Tables are NOT for layout.

Code Example

```
<table>
  <caption>Student Scores</caption>
  <thead>
    <tr>
      <th scope="col">Name</th>
      <th scope="col">Math</th>
      <th scope="col">Average</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Alice</th>
      <td>95</td><td>91.5</td>
    </tr>
  </tbody>
  <tfoot>
    <tr><th>Class Average</th><td>86.5</td><td>86.5</td></tr>
  </tfoot>
</table>
```

Tip: *Use scope="col" on column headers and scope="row" on row headers for screen reader accessibility.*

Chapter 19: Lists

Definition

HTML has three list types: <ul> (unordered / bulleted), <ol> (ordered / numbered), and <dl> (description list of term-definition pairs). Lists can be nested inside each other.

Code Example

```
<ul>
  <li>Apples</li>
  <li>Bananas</li>
</ul>

<ol type="A" start="3">
  <li>Third item as C</li>
</ol>

<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language.</dd>
```

```
  <dt>CSS</dt>
  <dd>Cascading Style Sheets.</dd>
</dl>
```

📖 **Tip:** *type="A" → A,B,C | type="I" → I,II,III | type="i" → i,ii,iii*

Media & Graphics

Focus: Visuals and interactive elements

Chapter 20: Images

Definition

The <img> element embeds images. It is a void element (self-closing). The src attribute provides the image source, and alt provides accessible alternative text.

Code Example

```
<img src="photo.jpg" alt="A scenic mountain" width="800" height="600">

<!-- Responsive image with srcset -->
<img src="photo-300.jpg"
  srcset="photo-300.jpg 1x, photo-600.jpg 2x"
  alt="Responsive photo">

<!-- Picture element -->
<picture>
  <source media="(min-width:800px)" srcset="large.jpg">
  <source media="(min-width:400px)" srcset="medium.jpg">
  <img src="small.jpg" alt="Responsive picture">
</picture>
```

Tip: *Always include a meaningful alt attribute. Empty alt="" is correct for decorative images.*

Chapter 21: Image Maps

Definition

An image map defines clickable areas (hotspots) on an image. Each area can act as a separate hyperlink. Bind the image to the map with the usemap attribute.

Code Example

```
<img src="world.png" alt="World Map"
  usemap="#worldmap" width="800" height="400">

<map name="worldmap">
  <area shape="rect"
    coords="50,80,200,200" href="europe.html" alt="Europe">
  <area shape="circle"
    coords="400,180,60" href="africa.html" alt="Africa">
  <area shape="default" href="other.html" alt="Other">
</map>
```

Tip: *Use an image map editor tool (like image-map.net) to easily calculate pixel coordinates for complex shapes.*

Chapter 22: Canvas

Definition

The <canvas> element is an HTML5 drawing surface. It is a container only — all drawing is done with JavaScript via the 2D rendering context (getContext("2d")).

Code Example

```
<canvas id="myCanvas" width="400" height="200"
  style="border:1px solid #ccc;">
  Canvas not supported.
</canvas>
<script>
  const ctx = document.getElementById("myCanvas")
                      .getContext("2d");
  ctx.fillStyle = "red";
  ctx.fillRect(10, 10, 150, 100);
  ctx.fillStyle = "black";
  ctx.font = "20px Arial";
  ctx.fillText("Hello Canvas!", 120, 170);
</script>
```

Tip: *Canvas is pixel-based and resolution-dependent. For scalable graphics, prefer SVG.*

Chapter 23: SVG (Scalable Vector Graphics)

Definition

SVG is an XML-based format for scalable, resolution-independent vector graphics. Unlike raster images (PNG/JPEG), SVG images remain crisp at any size.

Code Example

```
<svg xmlns="http://www.w3.org/2000/svg"
  width="200" height="200" viewBox="0 0 200 200">
  <circle cx="100" cy="100" r="80" fill="#1565C0"/>
  <rect x="60" y="60" width="80" height="80"
    fill="white" opacity="0.5"/>
  <text x="100" y="108" text-anchor="middle"
    fill="white" font-size="18">SVG!</text>
</svg>

<!-- External SVG as image -->
<img src="icon.svg" alt="Icon" width="50" height="50">
```

Tip: *Inline SVG gives you full CSS and JavaScript control. Use SVG for icons, logos, and charts on HiDPI/Retina screens.*

Chapter 24: Embed

Definition

The <embed> element (HTML5) embeds external content such as plugins, PDFs, or multimedia directly into the page. The type attribute must specify the MIME type.

Code Example

```
<embed type="video/mp4" src="video.mp4"
  width="640" height="480">

<embed type="application/pdf" src="document.pdf"
  width="800" height="600">

<embed type="image/svg+xml" src="graphic.svg"
  width="300" height="300">
```

📖 **Tip:** *Prefer <video>, <audio>, or <object> for media. Use <embed> only when integrating legacy plugin-based content.*

Chapter 25: Media Elements

Definition

HTML5 introduced native <audio> and <video> elements for embedding media without plugins. Multiple <source> children provide fallback formats for browser compatibility.

Code Example

```
<audio controls>
  <source src="song.ogg" type="audio/ogg">
  <source src="song.mp3" type="audio/mpeg">
  Your browser does not support audio.
</audio>

<video width="640" height="360" controls poster="thumb.jpg">
  <source src="video.webm" type="video/webm">
  <source src="video.mp4" type="video/mp4">
  <track kind="subtitles" src="subs-en.vtt"
    srclang="en" label="English" default>
</video>
```

📖 **Tip:** *Always provide multiple formats (mp4 + webm) for maximum browser compatibility. Use muted for autoplay.*

Forms & User Input

Focus: Collecting and handling data

Chapter 26: Forms

Definition

The <form> element groups input controls and manages submission of user data to a server. The action attribute sets the destination URL; the method attribute sets GET or POST.

Code Example

```
<form action="/submit" method="post" autocomplete="on">
  <fieldset>
    <legend>Personal Info</legend>
    <label for="fname">First Name:</label>
    <input type="text" id="fname" name="firstname" required>
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>
  </fieldset>
  <button type="submit">Submit</button>
  <button type="reset">Clear</button>
</form>
```

📖 **Tip:** *GET appends data to the URL (bookmarkable). POST sends data in the request body (more secure). Use POST for sensitive data.*

Chapter 27: Input Control Elements

Definition

The <input> element creates interactive form controls. Its type attribute determines behavior: text, email, password, checkbox, radio, file, number, range, date, color, and more.

Code Example

```
<input type="text" placeholder="Name" name="name">
<input type="email" placeholder="Email">
<input type="password" placeholder="Password">
<input type="number" min="1" max="100" value="10">
<input type="range" min="0" max="100" value="50">
<input type="date">
<input type="color" value="#1565C0">
<input type="checkbox" id="agree"> <label for="agree">I agree</label>
<input type="file" accept="image/*" multiple>
<input type="text" required minlength="3" maxlength="50">
```

📖 **Tip:** *HTML5 input types provide built-in validation and specialized keyboards on mobile (e.g., type="email" shows @ on mobile).*

Chapter 28: Label Element

Definition

The <label> element associates a text description with a form control. Clicking the label focuses or activates its associated input. This is critical for accessibility.

Code Example

```
<!-- Method 1: for attribute links to input id -->
<label for="username">Username:</label>
<input type="text" id="username" name="username">

<!-- Method 2: wrapping (implicit association) -->
<label>
  Password:
  <input type="password" name="password">
</label>

<!-- Labels with radio buttons -->
<fieldset>
  <legend>Contact method:</legend>
  <label><input type="radio" name="c" value="email"> Email</label>
  <label><input type="radio" name="c" value="phone"> Phone</label>
</fieldset>
```

Tip: *Every form input should have a <label>. This improves accessibility (screen readers) and usability (larger click target).*

Chapter 29: Selection Menu Controls

Definition

<select> creates a dropdown menu; <option> defines each choice; <optgroup> groups related options. The <datalist> element provides autocomplete suggestions for an <input> field.

Code Example

```
<select name="country">
  <option value="">-- Select --</option>
  <option value="us">United States</option>
  <option value="in" selected>India</option>
</select>

<!-- Option groups -->
<select name="cuisine">
  <optgroup label="Asian">
    <option value="japanese">Japanese</option>
  </optgroup>
  <optgroup label="European">
    <option value="italian">Italian</option>
  </optgroup>
</select>

<!-- Datalist autocomplete -->
<input list="browsers" placeholder="Choose browser">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
</datalist>
```

Tip: *datalist is not fully supported in all Safari versions. Use a polyfill for full cross-browser autocomplete support.*

Chapter 30: Output Element

Definition

The <output> element represents the result of a calculation or user action. It is associated with form inputs via the for attribute.

Code Example

```
<form oninput="result.value = parseInt(a.value) + parseInt(b.value)">
  <label>Number A: <input type="number" id="a" value="0"></label>
  +
  <label>Number B: <input type="number" id="b" value="0"></label>
  =
  <output name="result" for="a b">0</output>
</form>

<!-- Output with range slider -->
<form oninput="vol.value = volume.value">
  Volume: <input type="range" id="volume" min="0" max="100" value="50">
  <output name="vol" for="volume">50</output>
</form>
```

📖 **Tip:** *The <output> element updates live via form events. Use oninput on the <form> for real-time recalculation as users type.*

Chapter 31: Progress Element

Definition

The <progress> element displays a progress bar showing task completion. The value attribute is the current progress; max is the total. Without value, it shows an indeterminate/animated state.

Code Example

```
<!-- Determinate progress -->
<progress value="22" max="100">22%</progress>

<!-- Indeterminate (loading, unknown duration) -->
<progress max="100"></progress>

<!-- JavaScript-driven progress bar -->
<progress id="bar" value="0" max="100">0%</progress>
<button onclick="startProgress()">Start Task</button>
<script>
function startProgress() {
  let bar = document.getElementById("bar"), val = 0;
  let t = setInterval(() => {
    val += 5; bar.value = val;
    if (val >= 100) clearInterval(t);
  }, 200);
}
</script>
```

📖 **Tip:** *Use <meter> (not <progress>) to display a static gauge (like disk usage). <progress> is specifically for task completion.*

Advanced Features

Focus: Enhancing accessibility & interactivity

Chapter 32: ARIA (Accessible Rich Internet Applications)

Definition

ARIA adds semantic meaning to HTML elements so assistive technologies (like screen readers) can describe interactive components to users with disabilities. Use native HTML elements first; add ARIA only when needed.

Code Example

```
<!-- Alert -->
<div role="alert" aria-live="assertive">
  Your session expires in 60 seconds.
</div>

<!-- Dialog / Modal -->
<div role="dialog" aria-labelledby="dlgTitle">
  <h2 id="dlgTitle">Confirm Delete</h2>
  <button>Yes</button> <button>No</button>
</div>

<!-- Navigation landmark -->
<nav role="navigation" aria-label="Main menu">
  <ul>
    <li><a href="/">Home</a></li>
  </ul>
</nav>
```

Tip: *Do NOT add role="button" to a <button> — native elements already have implicit ARIA roles.*

Chapter 33: HTML Event Attributes

Definition

Event attributes attach inline JavaScript handlers to HTML elements that execute when specific events occur (click, input, submit, keydown, etc.).

Code Example

```
<!-- Form events -->
<input type="text"
  onfocus="this.style.background='lightyellow'"
  onblur="this.style.background=''"
  oninput="document.getElementById('out').textContent=this.value">
<p id="out"></p>

<!-- Mouse events -->
<button onclick="alert('Clicked!')"
  onmouseover="this.style.color='blue'"
  onmouseout="this.style.color=''">
  Hover and Click Me
</button>
```

📖 **Tip:** *Prefer addEventListener() in JavaScript over inline event attributes — it keeps HTML and JS separate and allows multiple handlers.*

Chapter 34: Include JavaScript Code in HTML

Definition

JavaScript can be included in HTML three ways: inline in a <script> tag, in an external .js file via the src attribute, or asynchronously/deferred to control load order.

Code Example

```
<!-- External script (best practice) -->
<script src="app.js"></script>

<!-- Inline script -->
<script>
  console.log("Page loaded!");
</script>

<!-- Async: downloads in parallel, executes immediately -->
<script src="analytics.js" async></script>

<!-- Defer: downloads in parallel, executes after parse -->
<script src="app.js" defer></script>

<!-- Module script (ES6) -->
<script type="module" src="main.mjs"></script>
```

📖 **Tip:** *Use defer for most scripts. Use async for independent scripts (analytics, ads). Modules are deferred by default.*

Chapter 35: HTML5 Cache (AppCache)

Definition

The HTML5 Application Cache (manifest) lets browsers store specified files offline. Note: AppCache is deprecated — use Service Workers in modern apps.

Code Example

```
<!-- index.html: reference the manifest -->
<!DOCTYPE html>
<html manifest="app.appcache">
<body>
  <h1>Offline-Capable Page</h1>
</body>
</html>

<!-- app.appcache manifest file -->
CACHE MANIFEST
# Version 1.0
CACHE:
  index.html
  style.css
NETWORK:
  api/data
FALLBACK:
  / offline.html
```

📖 **Tip:** *AppCache is deprecated. For production offline apps, use Service Workers with the Cache API (modern standard).*

Chapter 36: IFrames

Definition

An <iframe> (inline frame) embeds another HTML document inside the current page. Common uses include embedding maps, videos, and third-party widgets. Subject to the Same-Origin Policy for JavaScript access.

Code Example

```
<!-- Basic iframe -->
<iframe src="https://example.com" width="800" height="400"></iframe>

<!-- Embed a YouTube video -->
<iframe width="560" height="315"
  src="https://www.youtube.com/embed/VIDEO_ID"
  allowfullscreen>
</iframe>

<!-- Sandboxed iframe -->
<iframe src="untrusted.html"
  sandbox="allow-scripts allow-same-origin">
</iframe>
```

📖 **Tip:** *The sandbox attribute disables scripts, forms, and popups by default. Re-enable specific permissions using allow-scripts.*

Specialized Markup

Focus: Semantic enrichment

Chapter 37: Marking Up Computer Code

Definition

Use <code> for inline code snippets and <pre><code> for multi-line code blocks. Related elements: <var> for variables, <samp> for computer output, <kbd> for keyboard input.

Code Example

```
<p>Use the <code>console.log()</code> function to debug.</p>

<pre><code>
function greet(name) {
  return "Hello, " + name + "!";
}
</code></pre>

<p>The formula: <var>E</var> = <var>m</var><var>c</var><sup>2</sup></p>
<p>Output: <samp>Error: file not found</samp></p>
<p>Press <kbd>Ctrl</kbd> + <kbd>C</kbd> to copy.</p>
```

Tip: *Always escape < as < and > as > inside <code> blocks. Consider adding a syntax highlighter like Prism.js.*

Chapter 38: Marking Up Quotes

Definition

<q> is for short inline quotes; <blockquote> is for longer block-level quotes. The cite attribute on both specifies the source URL. The <cite> element marks the title of a creative work.

Code Example

```
<!-- Inline quote -->
<p>Einstein wrote
  <q>Imagination is more important than knowledge.</q>
</p>

<!-- Block quote with attribution -->
<blockquote cite="https://example.com/article">
  <p>The greatest glory in living lies not in never falling,
  but in rising every time we fall.</p>
  <footer>— <cite>Nelson Mandela</cite></footer>
</blockquote>

<!-- Cite element: title of a work -->
<p>I just finished reading <cite>The Great Gatsby</cite>.</p>
```

Tip: *Do NOT add quotation marks manually around <q> — browsers insert them automatically. <cite> is for titles, not author names.*

Chapter 39: Meta Information

Definition

<meta> tags in the <head> provide metadata about the document — character set, viewport, description, keywords, author, and social media sharing information.

Code Example

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Learn HTML from basic to advanced.">
  <meta name="author" content="Jane Developer">
  <meta name="robots" content="index, follow">

  <!-- Open Graph (Facebook / LinkedIn) -->
  <meta property="og:title" content="My Page Title">
  <meta property="og:image" content="https://example.com/image.jpg">

  <!-- Twitter Cards -->
  <meta name="twitter:card" content="summary_large_image">
  <meta name="theme-color" content="#1565C0">
</head>
```

📖 **Tip:** *The viewport meta tag is required for responsive design. Without it, mobile browsers will zoom out the desktop layout.*

Chapter 40: Void Elements

Definition

Void elements are HTML elements that cannot have any child content and therefore do not have a closing tag. HTML5 does not require the self-closing slash (/>) but it is allowed.

Key Points

- Complete list: <area> <base>
 <col> <embed> <hr> <img> <input> <link> <meta> <param> <source> <track> <wbr>

Code Example

```
<p>Line one.<br>Line two.</p>
<hr>

<img src="photo.jpg" alt="A beautiful landscape">
<input type="text" name="username" placeholder="Enter username">
<link rel="stylesheet" href="style.css">
<meta charset="UTF-8">

<!-- Word break opportunity -->
<p>superlongURL/<wbr>very/<wbr>deep/<wbr>path.html</p>
```

📖 **Tip:** *In HTML5,
 and
 are both valid. Self-closing syntax is required in XHTML/XML. Pick one style and be consistent.*